Bank System API Assessment

مستوى الامتحان	درجة التقييم	المدة المقترحة
Junior / Entry Backend	100 درجة	4 - 6 ساعات

وصف الحالة

مطلوب من الطالب بناء Backend بسيط لنظام بنكي مصغر يسمح للمستخدم بإدارة حسابه البنكي ومتابعة المعاملات المالية الخاصة به. الهدف ليس فقط تشغيل الـ APIs، لكن أيضًا كتابة كود نظيف، آمن، ومنظم مع مراعاة قواعد البزنس الأساسية.

المطلوب الأساسي

- استخدام js.Node مع Express أو NestJS.
- استخدام قاعدة بيانات SQL أو MongoDB مع تصميم واضح للعلاقات أو الـ references.
- تنفيذ Authentication بحيث كل مستخدم يرى حسابه ومعاملاته فقط.
- تنظيم المشروع بطريقة واضحة: routes / controllers / services / models / validations / middle ware.
- إرجاع responses منظمة مع status codes صحيحة ورسائل خطأ مفهومة.

الـ Models المطلوبة

الموديل	حقول أساسية	ملاحظات
User	id, fullName, email, password, role	كلمة السر تتخزن hashed
BankAccount	id, userId, accountNumber, balance, currency, status	كل مستخدم له حساب واحد على الأقل
Transaction	id, accountId, type, amount, balanceBefore, balanceAfter, status, createdAt	type مثل deposit / withdraw / transfer
Beneficiary	id, ownerUserId, accountNumber, bankName, nickname	بيانات المستفيد لتحويلات سريعة

الـ APIs المطلوبة

Endpoint	Method	الوصف	إجباري / Bonus
auth/register/	POST	إنشاء مستخدم جديد	إجباري
auth/login/	POST	تسجيل دخول وإرجاع token	إجباري
accounts/me/	GET	جلب بيانات حسابي البنكي الحالي	إجباري
transactions/my/	GET	جلب كل معاملاتي مع pagination	إجباري
transactions/:id/	GET	جلب معاملة واحدة مع التفاصيل	إجباري
transactions/deposit/	POST	إيداع مبلغ داخل الحساب	إجباري
transactions/withdraw/	POST	سحب مبلغ من الحساب	إجباري
transactions/transfer/	POST	تحويل لحساب آخر داخل نفس النظام	Bonus
transactions/my/summary/	GET	ملخص للإيداعات والسحوبات والرصيد الحالي	Bonus
accounts/me/statement/	GET	كشف حساب خلال فترة زمنية	Bonus

Business Rules مهمة

- لا يسمح بالسحب لو الرصيد غير كافٍ.
- قيمة المعاملة يجب أن تكون رقم موجب أكبر من صفر.
- أي عملية deposit أو withdraw يجب أن تسجل Transaction كاملة قبل/بعد الرصيد.
- المستخدم لا يستطيع الوصول لحساب أو معاملة تخص مستخدم آخر.
- في حالة التحويل: يتم الخصم والإضافة بشكل آمن، ويفضل داخل transaction على مستوى قاعدة البيانات إن أمكن.
- وجود status للمعاملة مثل pending / completed / failed.
- منع التعامل على الحساب لو status = frozen أو inactive.

التحقق والأمان

- Validation على body و params و query.
- استخدام hashing لكلمات المرور.
- JWT أو based auth-session.
- عدم إرجاع بيانات حساسة في الـ response مثل password.
- التعامل الجيد مع الأخطاء: insufficient balance, invalid transaction id, unauthorized access.

حاجات تتضاف كزيادة

- فلترة المعاملات حسب النوع أو التاريخ أو المبلغ.
- Pagination + sorting في قائمة المعاملات.
- Admin APIs لتجميد حساب.
- Postman docs.
- Rate limiting على endpoints.

معايير التقييم

تفاصيل التقييم	الدرجة	البند
هل التصميم منطقي وقابل للتوسع؟	20	تصميم الـ Models والعلاقات
هل كل المطلوب شغال بشكل صحيح؟	30	تنفيذ الـ APIs المطلوبة
هل السحب/الإيداع/التحويل متنفذين صح؟	20	Business Logic
JWT, hashing, input validation, authorization	15	Validation & Security
structure, naming, separation of concerns	10	تنظيم الكود والنظافة
tests, docs, docker, transfer, statement ...	5	Extras / Bonus

المطلوب في التسليم

- رابط GitHub
- README فيه خطوات التشغيل.
- لينك Collection Postman.
- seed data أو بيانات تجريبية لتسهيل الاختبار.